

✓ Flutter Interview Q&A

1. What is Flutter? 🚀

→ Flutter is an open-source UI toolkit developed by Google. It allows building beautiful, natively compiled applications for mobile, web, desktop, and embedded platforms—all from a single codebase.

2. Difference between Hot Reload and Hot Restart? ♻️

→ Hot Reload injects updated code into the Dart VM and refreshes UI instantly without losing state. Hot Restart rebuilds the widget tree from scratch, clearing all state and restarting the app.

3. What is the Widget Tree? 🌲

→ The widget tree represents the hierarchical structure of UI elements. Flutter uses this tree to efficiently build and update user interfaces based on state changes.

4. StatelessWidget vs StatefulWidget? ⚖️

→ **StatelessWidget** is immutable and doesn't change once built. **StatefulWidget** can rebuild itself when its internal state changes using a **State** object.

5. What is BuildContext? 🧱

→ **BuildContext** gives access to location in the widget tree, allowing widgets to access theme data, navigate, or locate ancestors using context.

6. What is the pubspec.yaml file used for? 📄

→ `pubspec.yaml` manages app metadata like name, version, dependencies, fonts, and assets. It's essential for package management in Flutter.

7. What is the difference between final and const? 🔒

→ `final` is a runtime constant (set once), while `const` is a compile-time constant. `const` can be used for deep immutability and widget optimization.

8. What is setState()? ↺

→ `setState()` notifies Flutter that the internal state has changed and triggers a UI rebuild of the affected widget.

9. What are keys in Flutter? 🔑

→ Keys preserve state when widgets move around in the tree. They help Flutter distinguish between widgets with similar structure but different identities.

10. What is the role of the main() function? 🎬

→ The `main()` function is the entry point of a Flutter app. It usually calls `runApp()` to inflate the widget tree.

11. What is FlutterEngine? 🧠

→ `FlutterEngine` is the runtime that executes Dart code, renders UI, and handles communication between Dart and platform code.

12. What is the difference between MaterialApp and WidgetsApp?



→ `MaterialApp` is a wrapper that provides material design components. `WidgetsApp` is a lower-level widget without material theming.

13. Explain Navigator and routing. 🗺️

→ The `Navigator` manages a stack of routes (screens) and allows navigation using `push`, `pop`, and `pushNamed` for named routes.

14. What is the difference between `async`, `await`, and `Future`? ⌚

→ `Future` represents a potential value. `async` makes a function asynchronous, and `await` pauses execution until the future completes.

15. What is `InheritedWidget`? 🧬

→ `InheritedWidget` allows data to be passed down the widget tree efficiently and accessed by descendants using `context`.

16. What are mixins in Dart? 🧰

→ Mixins allow code reuse in multiple classes without inheritance. They are declared using `with` keyword.

17. What is a stream in Dart? 🌊

→ A stream provides asynchronous data events over time, commonly used for listening to real-time data like user input or API responses.

18. How to manage state in Flutter? 📊

→ State can be managed using `setState`, `InheritedWidget`, `Provider`,

Bloc, Riverpod, GetX, MobX, etc., depending on app complexity.

19. What is Provider in Flutter? 🔥

→ **Provider** is a wrapper around `InheritedWidget` that simplifies state management by making it easier to inject and access state.

20. What is the difference between `mainAxisAlignment` and `crossAxisAlignment`? 📐

→ **`mainAxisAlignment`** aligns children along the main axis (horizontal in Row, vertical in Column). **`crossAxisAlignment`** aligns them perpendicularly.

21. What is FutureBuilder? 🕒

→ **FutureBuilder** builds UI based on the state of a **Future**. It shows different widgets while waiting, on error, or after completion.

22. Explain `ListView.builder`. 🧱

→ **`ListView.builder`** is used for long or infinite lists. It lazily builds list items as they scroll into view, improving performance.

23. What is the purpose of `SafeArea`? 📱

→ **SafeArea** ensures UI elements are not obscured by system notches, status bars, or curved screen edges.

24. How does Flutter handle gestures? 🖐️

→ Flutter uses **GestureDetector** and other gesture widgets to detect taps, swipes, drags, and other interactions.

25. Explain the lifecycle of StatefulWidget.

→ A `StatefulWidget` lifecycle includes: `createState()`, `initState()`, `didChangeDependencies()`, `build()`, `setState()`, `dispose()`.

26. What is the difference between runApp() and main()? 🚩

→ `main()` is the Dart entry point. `runApp()` attaches the root widget to the screen and starts rendering.

27. How does Flutter render UI?

→ Flutter uses Skia for rendering UI. It rebuilds widgets and repaints only affected parts, ensuring high performance.

28. What is CustomPainter?

→ `CustomPainter` allows custom drawing on the canvas, useful for creating shapes, graphs, or complex designs.

29. What is a GlobalKey?

→ A `GlobalKey` uniquely identifies a widget across rebuilds, enabling access to state or context from anywhere.

30. What are the types of Animations in Flutter?

→ Flutter supports implicit (`AnimatedContainer`) and explicit animations (`AnimationController` + `Tween` + `AnimatedBuilder`).

31. What is the difference between push and pushReplacement?



→ `push` adds a new route to the stack, while `pushReplacement` replaces the current route with a new one.

32. What are isolates in Dart?

→ Isolates are independent workers for parallel code execution in Dart, as Dart is single-threaded by default.

33. How do you optimize performance in Flutter?

→ Use const constructors, lazy building (`ListView.builder`), avoid rebuilding large trees, and profile with Flutter DevTools.

34. What is the use of `initState()`?

→ `initState()` is called once when the widget is inserted in the tree. It's used to initialize data and listeners.

35. What is the difference between `ModalRoute` and `PageRoute`?



→ `ModalRoute` is an abstract base class for routes that block interaction. `PageRoute` is a subclass with platform-specific transitions.

36. Explain Flutter DevTools.

→ DevTools is a suite of debugging tools for inspecting UI, tracking memory, CPU usage, and diagnosing performance issues.

37. What is the Scaffold widget?

→ `Scaffold` is a top-level Material widget that provides layout structure with AppBar, Drawer, BottomNavigationBar, etc.

38. What are extension methods in Dart?

→ Extension methods add functionality to existing classes without

modifying them. Useful for clean, reusable code.

39. What is the role of ThemeData? 🎨

→ `ThemeData` defines the visual theme of the app including colors, fonts, and icon styles applied app-wide.

40. What is null safety in Dart? ❓

→ Null safety helps prevent null reference errors by requiring variables to be non-nullable by default unless explicitly declared nullable.

41. What are some popular state management libraries? 📚

→ Provider, Riverpod, Bloc, Cubit, GetX, MobX, and Redux are commonly used for managing state in Flutter.

42. What is the use of the `mounted` property? 📌

→ `mounted` checks whether a state object is still in the widget tree before calling `setState()` to avoid errors.

43. What are some ways to persist data in Flutter? 💾

→ Use `SharedPreferences` for simple key-value storage, `SQLite` for structured data, or `Hive` for NoSQL-like local storage.

44. How to display a snackbar in Flutter? 🗉

→ Use `ScaffoldMessenger.of(context).showSnackBar()` to display a temporary message at the bottom of the screen.

45. What is a Cupertino widget? 🍏

→ Cupertino widgets provide iOS-style UI components like

CupertinoButton, CupertinoNavigationBar for native iOS look and feel.

46. What is the difference between Theme.of and MediaQuery.of?



→ Theme.of(context) gives theme data like colors and fonts.

MediaQuery.of(context) gives screen size, padding, and orientation.

47. What are slivers in Flutter?



→ Slivers are scrollable areas that provide custom scroll behaviors.

Used in CustomScrollView with SliverAppBar, SliverList, etc.

48. How to create responsive UI in Flutter?



→ Use LayoutBuilder, MediaQuery, or responsive packages like flutter_screenutil or responsive_framework.

49. What are the layers in Flutter's architecture?



→ Flutter has 3 layers: Framework (widgets), Engine (Skia, Dart runtime), and Embedder (platform integration like Android/iOS).

50. What's new in Flutter 3.x or 4.x?



→ Recent versions introduced Impeller renderer, Material 3, faster compilation, foldable support, and enhanced web & desktop performance.

🙌 Found this helpful?

👍 Like • 🔄 Repost • + Follow for more Flutter insights and tips!

Let's **ace your next Flutter interview**—together. 🧳🚀

🔥 *Stay tuned for more updates on Flutter, Dart, and modern app development.*